Project 2. Building a Simple Web Server

In this project, you will implement a simple Web server that serves file (called static content) requests to clients. This project will help you understand the Web or the Hypertext Transfer Protocol (HTTP) protocol, which is one of most popular application-level protocols on the Internet. Also, you will understand how a Web server works. For an extra credit, you can choose to implement it as an *event-driven* Web server.

shttpd: A Simple Web Server

We implement a Web server that supports a subset of the HTTP version 1.0 (HTTP/1.0). More specifically, the server needs to implement the following features:

- It needs to support only the `GET` method delivering file objects.
- It should support a persistent connection if a client asks for it. In a persistent connection, the server does not close the connection after serving one request: it reuses the same connection for serving subsequent requests until the client is done with it.
- It should support handling at least five requests simultaneously from multiple concurrent connections. This is the same requirement as the first programming assignment.
- You will get an extra credit if your server takes the event-driven architecture.

```
./shttpd -p 1080 -d /home/kyoungsoo/document
```

should start the server listening on TCP port 1080 with the document root directory for file search as "`/home/kyoungsoo/document`". We strongly recommend referring to RFC 1945 for HTTP/1.0 specification, but if there is any difference between this document and RFC 1945, you should follow this document (e.g., support for HTTP/1.1 requests, persistent connections, etc.)

How shttpd works?

Here are the operations of `shttpd`. We assume that a client has already established a connection with `shttpd`.

1. *Receive an HTTP GET request*: the server receives the entire request, and parses it. If the request is mal-formed, the server should send an error response (starting with "`HTTP/1.0 400 Bad Request`") and close the connection. The request line (i.e., the first line of the request) should start with

   ```
   GET URL HTTP-VER
   ```

   where `URL` should start with "`/`" (i.e., no need to support a URL starting with "`http://`") and `HTTP-VER` should be either `HTTP/1.0` or `HTTP/1.1`. This way, the server can handle both `HTTP/1.0` and `HTTP/1.1` `GET` requests. Allowing `HTTP/1.1` is convenient as existing client tools (like `wget` or `curl`) use `HTTP/1.1` by default. The request must contain the "`Host`" header field – otherwise, the server should treat the request as mal-formed. For the purpose of this assignment, the

server needs to (1) check if the "`Host`" header field exists (i.e., it does not need to validate the host name itself) and (2) handle only the "`Connection`" header field. It can ignore all other request header fields (e.g., no range queries, no If-Modified-Since, etc.). By default, a TCP connection for `HTTP/1.0` requests should be treated as *non-persistent* (or ephemeral) while the one for `HTTP/1.1` requests should be treated as *persistent*. If the "`Connection`" header field has "`Close`" (or "`Keep-Alive`") (case-insensitive) as value, then it means that the client wants the connection to be treated as non-persistent (or persistent). While a server does not need to honor this request in practice (e.g., due to security reasons, the server may disallow persistent connections no matter what), our server always honors it for the purpose of this assignment. The following two requests are handled identically as non-persistent connections.

```
GET /hello/world.txt HTTP/1.0
Host: www.example.com

GET /hello/world.txt HTTP/1.1
Host: www.example.com
Connection: Close
```

Similarly, the following two requests are handled identically as persistent connections.

```
GET /hello/world.txt HTTP/1.1
Host: www.example.com

GET /hello/world.txt HTTP/1.0
Host: www.example.com
Connection: keep-alive
```

2. *Find the requested file*: the server finds the requested file whose path consists of the document root directory concatenated with the request URL. For example, if the request is

   ```
   GET /hello/world.txt HTTP/1.0
   Host: www.example.com
   ```

   And say your server runs with `-d /home/kyoungsoo/document/`, then, it should find the file at `/home/kyoungsoo/document/hello/world.txt`. If the file does not exist, the server should send an error response (starting with "`HTTP/1.0 404 Not Found`") and close the connection.

3. *Send a response*: if everything is OK, the server sends a response. The response consists of one or more response header lines and a response body. The response

header part should contain both the "`Content-length`" and "`Connection`" header fields. The `Content-length` header field has the file size as the value. The "`Connection`" header field can have either "`Close`" or "`Keep-Alive`" (case-insensitive) to inform the client of its decision. The policy of `shttpd` is to follow the request of the client – if the client asks for persistent connection, the value should be "`Keep-Alive`". Otherwise, the value should be "`Close`". The response body should contain the content of the file that has been requested. The first response line should start with `HTTP/1.0` even if the request is `HTTP/1.1`. A valid response would look like

```
HTTP/1.0 200 OK
Content-length: 6554
Connection: close
(empty line)
[content of the requested file]
```

which means that the requested file has 6554 bytes and the server will close the connection after serving this file as requested by the client.

4. *Wait for the next request or close the connection*: When the server is done with sending the response, it should either wait for the next request on the persistent connection or close the connection if it is ephemeral. For the persistent connection, it should go back to step 1.

How to determine if a connection is persistent or ephemeral?

The server should determine whether to keep the connection persistent or ephemeral by parsing the request. If the request has the "`Connection`" header, then the server must honor what it suggests (ephemeral if the value is "`close`" or persistent if it is "`keep-alive`"). If the "`Connection`" header is missing in the request, then the server should treat an `HTTP/1.0` request as ephemeral and an `HTTP/1.1` request as persistent by default. Regardless of the request, the server must include the "`Connection`" header in all cases (even in an error response).

Event-driven Web Server Architecture

You can get up to **30%** your earned points as an extra credit if your server is implemented as an event-driven server. You can use either `epoll` (recommended due to performance advantage) or `select` for your implementation. Do not use any custom event library (e.g., `libevent`) as you might want to invoke the system call directly to understand the low-level detail about its usage. Even if your server is not event-driven, your server should handle at least 5 concurrent requests from multiple connections simultaneously. You can use the prefork approach explained in the first programming assignment.

Warning: Event-driven programming with non-blocking APIs and blocking API-based programming would entail completely different code architectures. In general, it's non-trivial to convert blocking API-based code into event-driven architecture code. So, we recommend

you make a decision on which programming architecture you will implement before starting to write the code.

Overview of `epoll`-based Programming

Event-driven programming sets up and waits for events, and handles the events when they arise. In `epoll`, `epoll_wait()` is the event monitoring system call.

```
int epoll_wait(int epfd, struct epoll_event *events, int
maxevents, int timeout);
```

It blocks until one or more of the events (`events`) for monitoring arise or a timeout occurs (plug in -1 if you want to wait for events indefinitely). In order to use the system call, (1) you need to create an `epoll` file descriptor (`epfd` above) with `epoll_create(int size)`. `size` is ignored in recent kernels, but it should be larger than 0. Then, (2) you need to set up which events you want to wait for as follows. All you need for the assignment would be read (`EPOLLIN`) and write (`EPOLLOUT`) events.

- A read event says that there is something to read about the socket fd. If the socket is a listening socket (as below), a read event occurs if there's at least one established connection that needs to be `accept()`ed. A read event with a regular connection socket means that you have data to read for the socket.
- A write event says that you can use the socket for the write operation. A write event with a regular connection socket means that the socket write buffer has non-zero space so that you would expect a positive return value with `write()` (or any similar write I/O calls like `send()`). Why is the write event needed? `write()` simply copies your user-level/application-level buffer data to a kernel-level socket write buffer, and returns the number of bytes that are copied. Then, the TCP stack in the kernel sends the data in the socket write buffer whenever it can – we will learn congestion/flow control that limits the send rate, later. The kernel-level socket write buffer has a limited space (e.g., a few hundreds of KB), so repeated calls (or even a single call with large data) of `write()` would eventually fill up the buffer. When you call `write()` when the socket write buffer is full, then it returns -1 with `errno` as `EAGAIN`. Then, you set up a write event with the socket and wait for it with `epoll_wait()`. Note that a write event is used for monitoring for the non-blocking `connect()` system call to finish, but you don't need it for this assignment as you write a server (no need to call `connect()`).

```
ev.events = EPOLLIN;

ev.data.fd = listen_sock;

if (epoll_ctl(epollfd, EPOLL_CTL_ADD, listen_sock, &ev) == -1)
{

    perror("epoll_ctl: listen_sock");

    exit(EXIT_FAILURE);

}
```

The above code snippet (copied and pasted from the `epoll`(7) man page) registers a read event for `listen_socket` to `epollfd`. If you want to unregister an event for monitoring, you can use the `EPOLL_CTL_DEL` option as the second argument. Please read the man page of `epoll_ctl()` for detail. The following code shows a typical event processing loop (the code is copied and pasted from the epoll(7) man page). `epoll_wait()` returns the number of events that have been triggered (-1 as error) and `events` would contain the set of the triggered events. Then, the code iterates through `events[n]` to see which socket has which event. The code shows handling a read event for a listening socket while it hides the code for a read event for regular connection sockets.

```
   for (;;) {
          nfds = epoll_wait(epollfd, events, MAX_EVENTS, -1);
          if (nfds == -1) {
              perror("epoll_wait");
              exit(EXIT_FAILURE);
           }

          for (n = 0; n < nfds; ++n) {
              if (events[n].data.fd == listen_sock) {
                   conn_sock = accept(listen_sock,
                               (struct sockaddr *) &addr,
&addrlen);
                   if (conn_sock == -1) {
                       perror("accept");
                       exit(EXIT_FAILURE);
                    }
                    setnonblocking(conn_sock);
                    ev.events = EPOLLIN;
                    ev.data.fd = conn_sock;
                    if (epoll_ctl(epollfd, EPOLL_CTL_ADD,
conn_sock,
                                  &ev) == -1) {
                        perror("epoll_ctl: conn_sock");
                        exit(EXIT_FAILURE);
                        }
```

```
                    } else {
                        do_use_fd(events[n].data.fd);
                    }
                }
            }
        }
```

General programming tip: Event-driven programming would require storing and retrieving the context for connection processing. So, you would need to define a *per-socket context* (typically implemented as a structure in C) that can be easily looked up with the socket descriptor when an event for the socket descriptor arises. You should assume that the write buffer can be full at any given time. So, if you want to send the data of 100 bytes in the buffer, but you could send only up to 99 bytes as the socket write buffer became full. Then you need to send the 100th byte next time when you get a write event for the socket. This is the same for the read case. Your `read()` may not read the entire request at one time. Say, the request ends with "`\r\n\r\n`", but only the request up to "`\r\n\r`" have arrived. Then, you need to register the read event to read more (it's also possible that the next byte is not '`\n`'). In the mean time, your per-socket context should keep the data as well as the number of bytes that have been read to receive the contiguous data. Event-driven programming is pretty asynchronous – Note that regular blocking API-based programming which is synchronous as the processing thread can keep the information as local variables in the function stack. This is why the former is called *stack-ripping*.

Testing with `curl`

You may want to test with a large file (e.g 1GB) to see if your event handling code works properly. You can use `curl` and `md5sum` to check the correctness of your `shttpd` for large file delivery.

Testing with `flexiclient` and `fileset`

The provided files include a few tools for testing robustness. These tools are built by Vivek S. Pai and I got the permission to use the tools for education. README has the information on how to use the tools. We suggest using

```
./single /fileXX | ./flexiclient -host SERVER_NAME -port
SERVER_PORT -active 5 -time X_SECONDS -exhdrs "Host:
SERVER_NAME\r\n" -persist force
```

for testing robustness. (Make sure your server can deliver `fileXX`.) This command would send the request for file `fileXX` repeatedly via 5 concurrent persistent connections. When you are done with basic testing, create a file set on server with

```
./fileset -s zipfset -n 100
```

which creates files under `/file_set/dirXXXXX/classY_Z`. Then, try testing with

```
./zipfgen -s spec -n 100 | ./flexiclient -host SERVER_NAME -port
SERVER_PORT -active 5 -time X_SECONDS -exhdrs "Host:
SERVER_NAME\r\n"
```

which would generate requests for different files that have been created by the `fileset` tool. Note that the tools provide different options (type the command without any parameter, and it will explain all the available options).

Collaboration Policy:

- You can discuss with other students, but you should write the code on your own – you should **NOT** look at someone else' code nor show/give your code to anyone else. You should NOT consult generative AIs for doing the homework – writing or debugging your code, etc. You can view and/or study generic network socket programming code or the code that explains the usage of a system/socket call.
- Please stay away from any kind plagiarism. If you are not sure about anything, ask TAs or the instructor.

Submission:

- We provide `macro.h`, and `Makefile`. Feel free to use `macro.h` or not. Do not modify either file when you submit them.
- Submit `shttpd.c, macro.h, Makefile,` and `readme.pdf` in a gzipped tar file format. Do **NOT** submit any other files. In `readme.pdf`, write (1) your detailed implementation strategy, especially on how your server supports concurrent connections, (2) how you tested your program, (3) any bugs in the current code, (4) all collaborators. The gzipped tar file should be "{YourID_without_dash}_{YourNameInEnglish}_assign2". If your student ID is 2024-12345 and your name is "Haechan Kim", then it should be `202412345_HaechanKim_assign2`. **Note that you should use your student ID without the dash (-).** Assume you have the files in the current directory:

  ```
  $ mkdir 202412345_HaechanKim_assign2
  $ cp shttpd.c macro.h readme.pdf Makefile
  202412345_HaechanKim_assign2
  $ tar zcf 202412345_assign2.tar.gz
  202412345_HaechanKim_assign2
  ```

  Alternatively, you may use the given script to make the tar file automatically.

  ```
  $ ./submit.sh 202412345 HaechanKim
  ```

Implementation hints:

- The request file size can be arbitrarily large (e.g., can be larger than 2GB). Using "`int`" is not enough to represent a file size. It should be OK to use (signed) "`long`" to represent a file size.
- You can use `sendfile()` for the assignment. `sendfile()` allows zero-copy as you don't need to read the file content into a user-level buffer before sending it out to the socket. With `sendfile()`, the kernel reads the file content into kernel memory

pages and uses them to send out the data. This saves memory copies (i.e., file content from kernel memory pages to a user-level buffer and from the user-level buffer to a kernel-level socket buffer) and kernel-user mode change. Note that the "`count`" argument in `sendfile()` has a limit (2GB), but in practice, you might want to use a much smaller value (e.g., 1MB) for it. Otherwise, `sendfile`() could fail due to lack of resources especially if you handle many concurrent requests.

- Assume the maximum header field length is 1024 bytes. You can print out an error message ("`400 Bad request`") and stop if the header is larger than 1024 bytes.
- `SIGPIPE` is generated if you call `read()` or `write()` on a connection that is already closed by the other endpoint. Your application would get killed if it does not handle the signal. We recommend to call `signal(SIGPIPE, SIG_IGN);` early on to ignore the signal.

Late Policy:

- We will not accept late submission. Any late submission will result in zero credit. We may consider extension if you are in extraordinary circumstances (e.g., hospitalization). Please inform us as soon as you know.

Grading:

- Integrity of the submission file - 5 pts
  - Submission file format, all required files, no modification of macro.h/Makefile, no additional files, etc)
- Content of `readme.pdf` - 10 pts
  - All required information
- Coding style and compilation issues - 5 pts
  - Proper indentation and comments, proper variable naming, remove unnecessary global variables, no compilation warnings in, etc.
  - Please note that you will get zero credit if compilation fails. We will not allow you to fix any errors in your code once it's submitted.
- `curl` test - 60 pts
  - HTTP/1.0 ephemeral - 10 pts
  - HTTP/1.1 ephemeral - 10 pts
  - HTTP/1.0 persistent - 10 pts
  - HTTP/1.1 persistent - 10 pts
  - Concurrent HTTP/1.0 ephemeral - 10 pts
    - Your `shttpd` should handle at least 5 concurrent connections.
  - Handling error (404/403 and 400) - 10 pts
- `flexiclient` test - 20 pts
  - `flexiclient` single within 5 concurrent connections - 10 pts
  - `flexiclient` zipfian within 5 concurrent connections - 10 pts
- If your code does not compile with our Makefile, you'll get zero credit. However, if it is a simple error (e.g., forgot to include a header file, uses a custom macro defined by you in your `macro.h`, etc.) that can be easily fixed, we can still grade it by fixing the error. In this case, you will get 10% of deduction as penalty on your earned points. So, be careful!

- (Optional) **+30%** pts of what you earned above, when you successfully implemented an event-driven (and of course non-blocking) shttpd server.

==========

Hands-on Experience with HTTP

(The following text is borrowed from the EE323 course at KAIST)

It's fairly easy to see this process in action without using a Web browser.

1. From a Unix prompt or a Linux terminal, type "telnet www.google.com 80". It opens a TCP connection to the server at www.google.com listening on port 80 - the default HTTP port.
2. Issue a request by typing "`GET / HTTP/1.0`".
3. Hit the enter twice and see what is happening.

What you are seeing is exactly what your Web browser sees when it goes to the Google homepage: the HTTP status line, the response header fields, and finally the HTTP message body - consisting of the HTML that your browser interprets to create a web page.